LEARN BEING FORWARD

Programming — JavaScript Programming

JS fundamentals & ES6+ syntax

COMPLETE LEARNING GUIDE

What You'll Learn

JavaScript is the one language that runs natively in every browser and, via Node.js, on servers and command lines too — which means its syntax is worth learning precisely rather than by vague analogy to other languages, since a surprising number of its behaviors (type coercion, loose equality, var's scoping) differ from what experience in other C-family languages would predict. This sub-module covers modern (ES6 and later) JavaScript as it's actually written today: block-scoped variables, the primitive type system and coercion rules, template literals, destructuring, spread/rest syntax, default parameters, ES modules, and the newer optional chaining and nullish coalescing operators — building the vocabulary every later topic (functions, async code, the DOM) is written in.

1. Variables and Block Scope: let, const, var

var is function-scoped (or global-scoped if declared outside any function) and hoisted with its declaration initialized to undefined, meaning it's technically accessible — though unassigned — before its declaration line runs; this produces confusing bugs especially inside loops with asynchronous callbacks. let and const are block-scoped, confined to the nearest enclosing {}, and live in a temporal dead zone from the top of their block until their declaration line, so accessing them early throws a clear ReferenceError instead of silently returning undefined. const prevents the variable binding from being reassigned, but does not freeze the value itself — a const array or object can still have its contents mutated.

JAVASCRIPT

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // logs 3, 3, 3 — one shared `i`
}

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0); // logs 0, 1, 2 — a fresh `j` per
iteration
}

const config = { retries: 3 };
config.retries = 5; // legal — mutating the object, not reassigning `config`
```

2. Primitive Types, typeof, and Type Coercion

JavaScript has seven primitive types — string, number, boolean, undefined, null, symbol, and bigint — plus object, which arrays, functions, and dates are all specialized forms of. typeof reports a value's type at runtime but has two long-standing quirks worth memorizing: typeof null returns 'object' (a bug preserved for backward compatibility since the language's earliest versions) and typeof on an array also returns 'object' — use Array.isArray() to actually distinguish an array. The loose equality operator == coerces operands to a common type before comparing, producing famously surprising results ('' == 0 is true); the strict equality operator === compares both type and value with no coercion at all, and should be the default choice in virtually all modern code.

- Falsy values: false, 0, -0, '', null, undefined, NaN. Everything else — including '0' and [] — is truthy.
- NaN is the only value never equal to itself, even to another NaN — use Number.isNaN(x), never x === NaN.

```javascript
console.log(typeof null);           // "object" (long-standing quirk)
console.log(Array.isArray([1, 2]));  // true — the correct way to check for an array
console.log('' == 0);                // true  — loose equality coerces
console.log('' === 0);               // false — strict equality does not
```

3. Template Literals and Modern String Methods

Template literals, written with backticks instead of quotes, support embedded expressions via ${...} interpolation and genuine multi-line strings without the \n and string-concatenation gymnastics older quoted strings required. Beyond interpolation, modern String methods cover most everyday text tasks directly: includes() for substring checks, startsWith()/endsWith(), padStart()/padEnd() for fixed-width formatting, trim() (plus trimStart()/trimEnd()), and replaceAll() for replacing every occurrence of a substring or pattern without needing a global regular expression flag.

```javascript
const name = "Asha";
const score = 92;
const message = `Hello ${name}, your score is ${score}/100.
Keep it up!`; // genuine multi-line string, no \n needed

console.log("file.TXT".endsWith(".txt".toUpperCase())); // true
console.log("5".padStart(3, "0"));                      // "005"
```

4. Destructuring Arrays and Objects

Destructuring unpacks values from arrays or properties from objects directly into named variables in a single expression, instead of accessing each one individually by index or key. Array destructuring matches by position (const [first, second] = arr), while object destructuring matches by property name (const { name, age } = user) and can rename a variable on the fly ({ name: userName }). Destructuring also works directly in function parameter lists, which is an extremely common pattern for functions that accept a single

options object.

```javascript
const point = [10, 20];
const [x, y] = point;

const user = { name: "Raj", age: 31, city: "Pune" };
const { name, age: userAge, country = "India" } = user; // rename + default value

function createUser({ name, age = 18 }) {
  return { name, age };
}
console.log(createUser({ name: "Priya" })); // { name: "Priya", age: 18 }
```

5. Spread and Rest Operators

The same ... syntax serves two opposite roles depending on context. As spread, it expands an array or object's elements/properties in place — copying an array into a new one, merging objects, or passing an array's elements as individual function arguments. As rest, it collects any remaining arguments or array/object entries into a single new array or object, commonly used to gather a variable number of function arguments or to split a few known object properties away from 'everything else'. Crucially, spread only performs a shallow copy — nested objects or arrays inside the copy still point at the same nested references as the original.

```javascript
const base = { theme: "dark", fontSize: 14 };
const override = { ...base, fontSize: 16 }; // spread: shallow copy + override

function sum(...numbers) { // rest: gathers all args into an array
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10

const { fontSize, ...restSettings } = override; // rest in destructuring
console.log(restSettings); // { theme: "dark" }
```

6. Default Parameters and Enhanced Object Literals

Function parameters can declare a default value (function greet(name = 'Guest')) that's used only when the argument is omitted or explicitly undefined — not when it's null or any other falsy value, a distinction worth remembering. Object literals support several shorthand conveniences: property shorthand ({ name } instead of { name: name } when the variable name matches the key), method shorthand ({ greet() {...} } instead of { greet: function() {...} }), and computed property names ({ [dynamicKey]: value }) for building a key from a runtime expression.

```javascript
function greet(name = "Guest") {
  return `Hello, ${name}!`;
}
console.log(greet());        // "Hello, Guest!"
console.log(greet(undefined)); // "Hello, Guest!" — default still applies
console.log(greet(null));    // "Hello, null!" — null does NOT trigger the default

const id = "score";
const stats = {
  id,                       // property shorthand
  [id + "_max"]: 100,       // computed property name
  describe() { return `Tracking ${this.id}`; }, // method shorthand
};
```

7. ES Modules: import and export

ES Modules let a file explicitly declare what it exposes to other files with export (either named exports, any number per file, or a single default export) and what it pulls in from elsewhere with import. Each module has its own top-level scope — a variable declared in one file is never implicitly global to another, which was a real problem with plain <script> tags before modules existed. In the browser, a module is loaded with <script type="module" src="app.js">; in Node.js, the same import/export syntax works in .mjs files or any file when the project's package.json sets "type": "module".

```javascript
// mathUtils.js
export const PI = 3.14159;
export function square(n) { return n * n; }
export default function cube(n) { return n ** 3; }

// app.js
import cube, { PI, square } from "./mathUtils.js";
console.log(square(4), cube(2), PI); // 16 8 3.14159
```

8. Optional Chaining and Nullish Coalescing

Optional chaining (?.) short-circuits to undefined instead of throwing when accessing a property on something that might be null or undefined — user?.address?.city returns undefined safely at any missing link in the chain rather than crashing with 'cannot read property of undefined'. A matching ?.() syntax calls a function only if it exists. The nullish coalescing operator (??) supplies a default only when the left side is specifically null or undefined — unlike ||, which falls back on every falsy value, incorrectly overriding legitimate values like 0, '', or false.

- user?.profile?.bio ?? 'No bio provided' safely handles a missing profile AND distinguishes an intentionally empty bio from a missing one if written carefully.

- count || 10 wrongly replaces a real count of 0 with 10; count ?? 10 correctly keeps 0 and only replaces null/undefined.

```javascript
const user = { profile: null };
console.log(user.profile?.bio);         // undefined — no crash
console.log(user.profile?.bio ?? "No bio provided"); // "No bio provided"

const settings = { volume: 0 };
console.log(settings.volume || 50);  // 50 — WRONG, 0 is valid but falsy
console.log(settings.volume ?? 50);  // 0  — correct, only null/undefined fall back
```

Common Pitfalls

- Using var out of habit, letting a variable leak out of an if/for block's scope unexpectedly since var ignores block boundaries.
- Confusing ?? (only replaces null/undefined) with || (replaces every falsy value), accidentally overriding a legitimate 0, '', or false default.
- Assuming destructuring a missing property throws an error — it silently yields undefined, which can mask a typo in the property name.
- Expecting the spread operator to deep-clone a nested object or array — it only performs a shallow copy, so nested references are still shared.
- Mixing default and named exports inconsistently across files, leading to import syntax that differs confusingly from file to file.
- Forgetting the matching ?.() call syntax when optional-chaining a function that might not exist, instead of just ?. on the property access.
- Passing null to a parameter with a default value, expecting the default to kick in — defaults only apply to undefined, not null.

Key Takeaways

- Prefer const by default, let only when a variable will be reassigned, and avoid var entirely in new code.
- === (and ??) avoid the surprising coercion that == and || silently perform.
- Destructuring, spread, and rest all share the ... /pattern-matching syntax family but solve different problems — unpacking, expanding, and collecting, respectively.
- Default parameters trigger only on undefined, never on null or other falsy values.
- ES Modules give each file its own scope — nothing is implicitly global the way old-style <script> tags were.
- Optional chaining (?.) prevents crashes on missing nested properties; nullish coalescing (??) supplies defaults only for null/undefined.

Learn More

MDN — JavaScript Guide
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

MDN — Destructuring assignment
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment

javascript.info — The Modern JavaScript Tutorial
https://javascript.info/